

Adda

১৫০০, ১০০০০০ ১০০০০০০

Professional Communication Web App — A to Z Product & Engineering Blueprint

এক **১৫০০** Messenger/WhatsApp-**১০০০০০** **১০০০০০** modern, secure, scalable **১০০০** unique communication platform **১০০০০০** **১০০০০** product vision, feature set, UI/UX, frontend, backend, database, real-time architecture, security, deployment, testing **১০০০** future mobile/PWA roadmap **১০০০০০** **১০০০০০** **১০০০০০**

1. Product Vision

Brand: Adda | Tagline: **১৫০০**, **১০০০০০** **১০০০০০০**

Adda **১৫০০** **১৫০০** **১৫০০** communication platform **১৫০০০০** private chat, group/community, voice/video calling, media sharing, reactions, presence **১৫০০** privacy-first controls **১৫০০০০**—**১৫০০০০** UI/UX **১৫০০** identity-**১৫০০** Bangla-first **১৫০০০** personality **১৫০০০০**

Core idea: “Simple to start, powerful when needed.”

2. What Makes Adda Unique

- Bangla-first identity: **১৫০০০০** **১৫০০**, **১৫০০০০** onboarding, Bangla/English language switch **১৫০০** culturally familiar microcopy
- Adda Rooms: temporary **১৫০০** persistent group rooms **১৫০০০০** chat, pinned topics, files, polls **১৫০০** shared media **১৫০০** **১৫০০০০** **১৫০০০০**
- Smart Conversation Mode: message reply, thread, mention, poll, checklist **১৫০০** event-style messages
- Privacy Center: active sessions, device management, disappearing messages, block/report, privacy controls **১৫০০** account export/delete flow
- Message States: sending → sent → delivered → seen; retry **১৫০০** offline queue **১৫০০** reliable UX
- Conversation Search: global search, people, messages, files, links **১৫০০** date filters
- Personalization: chat wallpaper/theme, notification controls, compact/cozy density, custom status
- Web-first PWA: browser **১৫০০০** **১৫০০০০০০**, **১৫০০** installable app; responsive mobile experience
- Future-ready architecture: WebSocket layer, Redis, PostgreSQL **১৫০০** object storage **১৫০০** **১৫০০** scale **১৫০০** **১৫০০০০**

3. Recommended Technology Stack

Layer	Recommended choice	Purpose
Frontend	Next.js 14 + App Router	SSR/SEO, routing, web UI
Language	TypeScript + React 18	Type safety + component architecture
Styling	Tailwind CSS	Fast responsive UI
Icons	Lucide React	Consistent icon system
Frontend utilities	clsx + tailwind-merge	Conditional/class composition

Layer	Recommended choice	Purpose
Backend	Python 3.11 + FastAPI	REST API + real-time services
ASGI	Uvicorn	FastAPI runtime
Auth	JWT + bcrypt	Authentication + password hashing
Validation	Pydantic v2	Request/response validation
HTTP client	HTTPX	Service-to-service/external API calls
ORM	SQLAlchemy 2.0 Async	Async DB access
Dev DB	SQLite +aiosqlite	Local development / prototype
Production DB	PostgreSQL	Reliable concurrent production database
Real-time	WebSocket	Live chat/presence/typing
Cache/pub-sub	Redis	Presence, rate limit, pub/sub, scaling
Media	S3-compatible object storage	Images/videos/files
Deployment	Docker + reverse proxy	Repeatable production deployment

4. System Architecture

Browser/PWA → Next.js UI → FastAPI REST API + WebSocket Gateway → PostgreSQL/SQLite + Redis + Object Storage. Authentication and authorization sit between the client and API. WebSocket handles real-time events; REST handles account, conversation, history, search and settings operations.

Recommended production flow: Client → CDN/Reverse Proxy → Next.js → FastAPI → PostgreSQL. WebSocket connections are routed to the real-time service, with Redis Pub/Sub when multiple backend instances are running.

5. Frontend Structure

- App shell: sidebar, chat list, main conversation panel, details drawer, mobile navigation.
- Authentication pages: welcome, sign up, login, OTP/email verification if enabled, forgot password.
- Chat UI: message bubbles, grouping by sender/time, reactions, replies, attachments, link previews, pinned messages.
- Composer: text, emoji, attachment, voice-message placeholder, reply mode, edit mode, mention autocomplete.
- States: loading skeleton, empty state, offline state, reconnecting state, failed message retry.
- Accessibility: keyboard navigation, visible focus, semantic buttons, screen-reader labels and sufficient contrast.
- Responsive design: desktop 3-column layout; tablet 2-column; mobile single-panel navigation.

6. Main Pages / Screens

Landing / Welcome	Brand story, CTA, language, privacy promise.
Login / Register	Email/phone option, password, verification, session security.
Home / Inbox	Conversation list, search, filters, unread count.
Private Chat	1-to-1 messages, media, reactions, reply/edit/delete.

Group / Adda Room	Members, roles, pinned content, topics, polls.
Profile	Avatar, display name, bio/status, username.
Settings	Account, privacy, notifications, appearance, devices, security.
Search	People/messages/files/links with filters.
Media Viewer	Images/video/document preview and download.
Admin / Moderation	Reports, users, rooms, bans, audit logs.

7. Backend Modules

- Auth service: registration, login, refresh/revocation strategy, password reset, session/device management.
- User service: profile, username, avatar, presence, privacy settings, blocking.
- Conversation service: direct chats, groups, membership, roles, mute/archive/pin.
- Message service: send/edit/delete, replies, reactions, attachments, delivery/read receipts.
- Realtime gateway: WebSocket connection management, typing, presence and message events.
- Media service: upload authorization, metadata, validation, object-storage integration.
- Notification service: in-app notifications; web push can be added later.
- Search service: PostgreSQL full-text/trigram search or a dedicated search engine at larger scale.
- Moderation service: report, block, room controls, audit logs and admin actions.

8. Database Design

Your SQLite +aiosqlite setup is excellent for development. For production, use PostgreSQL. Keep SQLAlchemy models portable so migration is straightforward.

- users: id, username, display_name, email/phone, password_hash, avatar_url, bio, status, created_at
- sessions: id, user_id, refresh/session metadata, device_name, IP metadata, created_at, last_seen, revoked_at
- conversations: id, type, title, avatar_url, created_by, created_at, updated_at
- conversation_members: conversation_id, user_id, role, joined_at, muted_until, last_read_message_id
- messages: id, conversation_id, sender_id, body, type, reply_to_id, edited_at, deleted_at, created_at
- message_attachments: id, message_id, object_key, mime_type, size, width, height, duration
- message_reactions: message_id, user_id, emoji, created_at
- message_receipts: message_id, user_id, delivered_at, seen_at
- blocks: blocker_id, blocked_id, created_at
- reports: reporter_id, target_user/message/conversation, reason, status, reviewed_by
- notifications: id, user_id, type, payload, read_at, created_at
- devices/push_subscriptions: user/device/push endpoint metadata
- audit_logs: actor_id, action, target_type, target_id, metadata, created_at

9. Message & Real-Time Event Model

Use stable event names so frontend/backend remain easy to maintain.

Client → Server: message.send, message.edit, message.delete, typing.start, typing.stop, receipt.seen, presence.update

Server → Client: message.created, message.updated, message.deleted, typing.changed, presence.changed, receipt.updated, conversation.updated

For reliability, the client should generate a temporary `client_message_id`. The server returns the canonical message id. This prevents duplicate messages during reconnect/retry.

10. Authentication & Security

- Never store plaintext passwords; bcrypt/Argon2-style password hashing should be used.
- Use short-lived access tokens and a safer refresh/session strategy; revoke sessions on logout or suspicious activity.
- Validate every request with Pydantic and enforce authorization at the service layer.
- Use HTTPS in production and secure cookie/token handling appropriate to the architecture.
- Rate-limit login, OTP, message sending and upload endpoints.
- Validate file MIME type, size and extension; never trust client-provided filenames.
- Prevent XSS by safely rendering message content; sanitize or avoid arbitrary HTML.
- Use CSRF protection when authentication is cookie-based.
- Use CORS allowlists instead of wildcard origins in production.
- Keep secrets in environment variables/secret manager, not source control.
- Maintain audit logs for sensitive admin actions.

11. Media & File Sharing

- Do not store large files directly inside PostgreSQL/SQLite.
- Use object storage such as S3-compatible storage; database stores metadata and object keys.
- Generate thumbnails for images/videos and enforce upload limits.
- Use signed URLs or controlled download endpoints.
- Add virus/malware scanning before making files broadly available in a production environment.
- Create a background job system later for thumbnails, media processing and notification fan-out.

12. Unique UX / Product Features

- Addabari: a personal dashboard of favorite conversations/rooms.
- Quick Adda: one-tap temporary group room for a limited-time discussion.
- Conversation Topics: optional topic labels such as Study, Friends, Work, Project.
- Message Collections: save selected messages into private collections.
- Shared Board: a lightweight shared checklist/poll/event area inside rooms.
- Quiet Hours: schedule notification silence.
- Trusted Devices: visible device/session list with remote logout.
- Privacy Snapshot: a simple screen showing who can see profile, status and activity.
- Bangla + English smart UI copy with easy language switching.

13. Notifications

- Unread badge per conversation.
- Desktop/web push notifications through a service worker.
- Mention/reply notifications.
- Notification preferences per conversation.
- Do-not-disturb / quiet hours.
- Notification batching so high-volume rooms do not spam the user.

14. PWA / Installable Web App

Build Adda as a Progressive Web App so users can install it from supported browsers. Add a web app manifest, icons, service worker, offline shell/cache strategy and responsive layouts. Important: PWA installation is not identical to a native app; if later you need App Store/Play Store distribution, consider wrapping the web experience or building a dedicated mobile client.

15. Project Folder Structure

Frontend

app/ · components/ · features/ · hooks/ · lib/ · services/ · types/ · public/ · styles/

Backend

app/main.py · app/api/ · app/core/ · app/models/ · app/schemas/ · app/services/ · app/repositories/ · app/websocket/ · app/db/ · tests/

Keep business logic out of route handlers. Use service/repository layers so the codebase remains testable and maintainable.

16. API Blueprint

Suggested REST endpoints:

- POST /auth/register
- POST /auth/login
- POST /auth/refresh
- POST /auth/logout
- GET /users/me
- PATCH /users/me
- GET /users/{username}
- GET /conversations
- POST /conversations
- GET /conversations/{id}
- POST /conversations/{id}/members
- DELETE /conversations/{id}/members/{user_id}
- GET /conversations/{id}/messages
- POST /conversations/{id}/messages
- PATCH /messages/{id}
- DELETE /messages/{id}
- POST /messages/{id}/reactions
- DELETE /messages/{id}/reactions/{emoji}
- POST /media/upload-url
- GET /search?q=...
- GET /notifications
- PATCH /notifications/{id}/read

WebSocket: /ws?token=... or a secure authenticated handshake. Prefer a design where the server validates identity before subscribing the socket to conversation channels.

17. Development Roadmap

Phase 1 — Foundation	Repo setup, Next.js, FastAPI, SQLAlchemy, migrations, environment config, basic CI.
Phase 2 — Auth	Register/login, password hashing, sessions, profile.
Phase 3 — Core Chat	Conversations, messages, pagination, edit/delete, reply.
Phase 4 — Real-Time	WebSocket, delivery/read receipts, typing, presence, reconnect.
Phase 5 — Media	Image/file upload, previews, object storage.
Phase 6 — Groups	Adda Rooms, roles, member management, pinned messages.
Phase 7 — UX	Search, notifications, themes, responsive polish, accessibility.
Phase 8 — Security	Rate limits, audit logs, abuse controls, security review.
Phase 9 — Production	PostgreSQL, Redis, Docker, reverse proxy, monitoring, backups.
Phase 10 — PWA & Growth	Installability, push notifications, analytics, performance optimization.

18. Testing Strategy

- Backend unit tests: auth, message rules, permissions, validation.
- API integration tests: registration → login → create conversation → send message.
- WebSocket tests: reconnect, duplicate prevention, event delivery.
- Frontend component tests: composer, message list, conversation list.
- End-to-end tests: Playwright for major user journeys.
- Load testing: concurrent WebSocket connections, message throughput, DB query performance.
- Security tests: authorization bypass, rate limits, upload validation, XSS, session revocation.

19. Performance Plan

- Paginate messages; never load an entire conversation.
- Use cursor-based pagination for high-volume message history.
- Add DB indexes for conversation_id + created_at, sender_id, username and receipt lookups.
- Virtualize long message lists when necessary.
- Lazy-load images and media.
- Cache high-read data with Redis when scale requires it.
- Use background jobs for expensive work such as thumbnails and notifications.
- Monitor API latency, DB query time, WebSocket connection count and error rates.

20. Production Deployment

- Frontend: deploy Next.js on a suitable Node-compatible platform or container.
- Backend: Dockerized FastAPI/Uvicorn behind a reverse proxy/load balancer.
- Database: managed PostgreSQL with automated backups.
- Redis: managed Redis for cache/pub-sub/rate limiting.
- Storage: S3-compatible object storage for media.
- Domain: app.adda.example or similar subdomain.
- HTTPS: mandatory in production.
- Observability: structured logs, error tracking, uptime monitoring and metrics.

21. Admin Dashboard

- Overview: total users, active users, messages, rooms, reports.

- User management: search, profile, status, session/device actions.
- Room management: ownership, members, moderation settings.
- Reports queue: reported message/user/room, reason, evidence and resolution.
- Audit logs: admin actions with timestamp and target.
- System health: API status, DB status, WebSocket connections, storage usage.
- Do not expose sensitive credentials or raw password data to administrators.

22. MVP Feature List

- Registration/login
- Profile + avatar
- 1-to-1 chat
- Group chat
- Real-time messaging
- Typing indicator
- Online/offline presence
- Delivered/seen receipts
- Message edit/delete
- Reply + reactions
- Image/file sharing
- Search
- Notifications
- Block/report
- Responsive PWA UI

23. Future Version Features

- Voice/video calling using WebRTC
- Screen sharing
- Voice messages
- Message translation
- AI-assisted search/summarization (privacy-conscious and opt-in)
- Advanced community/room permissions
- Multi-device synchronization
- Native Android/iOS client if product traction justifies it
- End-to-end encryption only after designing and auditing the cryptographic protocol properly.

24. Important Architecture Decision: E2EE

If Adda eventually promises end-to-end encryption, do not implement custom cryptography. Treat E2EE as a separate security project using established, audited protocols/libraries. Server-side searchable content, moderation and encrypted message storage have different trade-offs; define the privacy model before promising E2EE publicly.

25. Recommended Build Order

Start small: Auth → Users → Conversations → Messages → WebSocket → Receipts/Typing → Media → Groups → Search → Notifications → Privacy → PWA → Production scaling. Do not start with voice/video calling, AI or complex encryption before the core messaging architecture is stable.

26. Definition of Done for v1

- User can register/login securely.
- User can create/find another user and start a chat.
- Messages appear in real time without refreshing.
- Reconnect does not silently lose messages.
- Message history loads efficiently.
- Images/files can be shared safely.
- Unread/read state is consistent.
- Group roles and permissions work.
- Block/report works.
- App is responsive on desktop and mobile.
- Production environment uses PostgreSQL, HTTPS, secrets management and backups.
- Basic monitoring, logging and automated tests are in place.

27. Final Product Direction

Adda should not try to win by copying every Messenger/WhatsApp feature. Build a focused communication product with a strong Bangla-first identity, clean UX, reliable real-time messaging, privacy controls and a modular architecture. Once the core is stable, add calling, richer rooms, PWA push notifications and other advanced features.

Tagline: বাংলা, বিশ্বের সাথে যোগাযোগ